
Plate-Local River Generation: Deterministic, Terrain-Aware, Downhill-Flowing Rivers for Infinite Procedural Worlds

Michael K. Davis III, Independent Researcher, USA
mikedavismk3@icloud.com

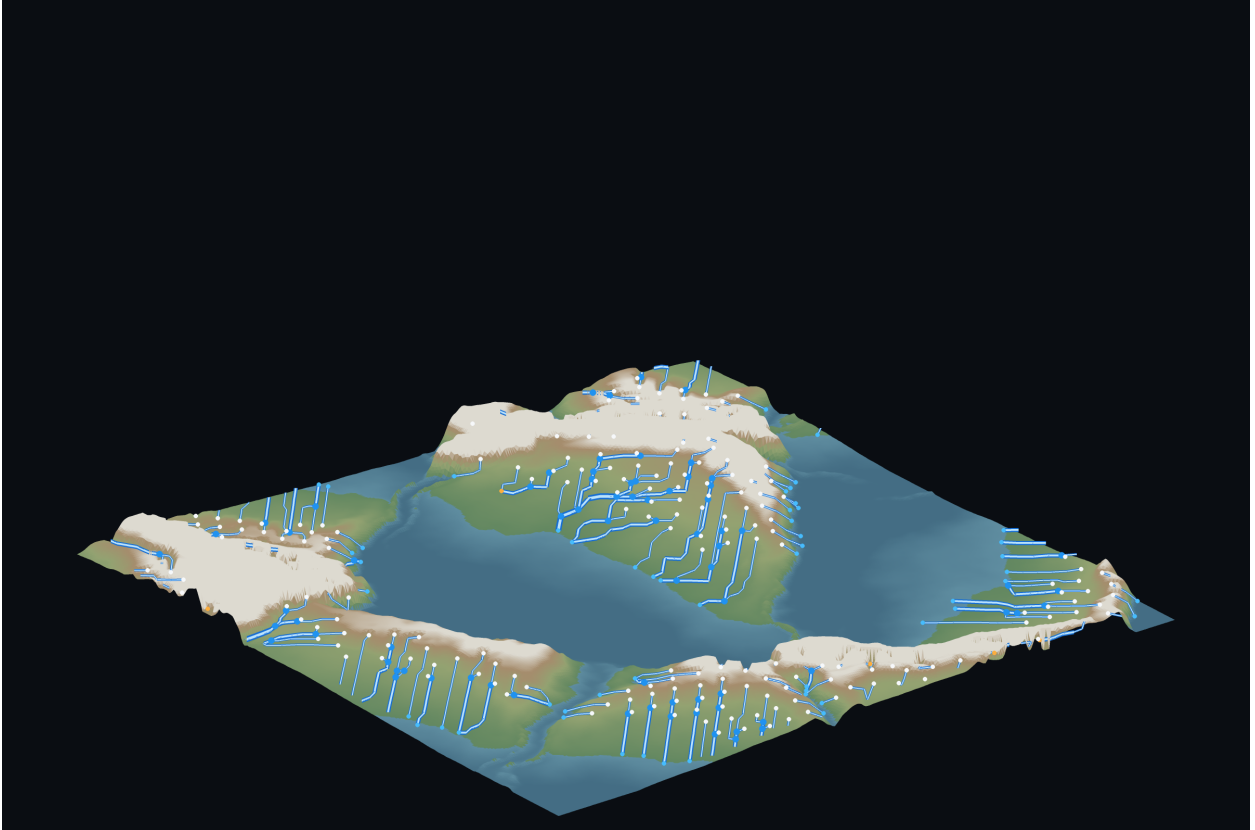


Figure 1: Plate-local river networks generated on demand across a regional view of the infinite-world implementation. Ten plate domains independently generate terrain-aware, downhill-flowing river networks without global preprocessing. River geometry is depth-tested against the terrain, hiding channels occluded by ridges. White dots indicate source nodes, blue indicate confluences, and yellow indicate lakes at local minima.

Abstract

Existing procedural terrain generation methods produce coherent, downhill-flowing river networks only on fixed domains, whereas approaches that extend to infinite worlds typically sacrifice network coherence or downhill-flow guarantees. This work presents a deterministic, terrain-aware, downhill-flowing river algorithm for infinite procedural worlds. Tectonic plate

seeds define both continuous terrain fields and corresponding bounded Voronoi-like plate domains, so localized river preprocessing can operate directly on the geometric domain without sampling terrain fields for the purpose of locating plate boundaries. Within each plate, the terrain field is sampled on a low-resolution grid, and rivers are traced downhill from highland source nodes to coastlines, local minima, or existing confluences. The resulting river network, together

with a geometric partition of the plate into nearest-river regions, is generated and cached lazily per plate, so each plate is processed independently of its neighbors. During full-resolution terrain sampling, a query point’s nearest river segment is located directly from its cached region, and river height is interpolated along that segment and blended with the locally evaluated terrain on demand—without caching the blended result itself—guaranteeing downhill flow. Together, these design choices make terrain-aware river generation practical for interactive procedural worlds by supporting efficient on-demand evaluation, as demonstrated by the performance analysis presented in this work.

1 Introduction

Procedural, deterministic, locally generatable terrain is well suited to large worlds because the terrain can be generated in independent chunks that remain seamless with neighboring chunks. However, large-scale features such as river networks may depend on terrain relationships that extend far beyond the chunk currently being generated.

This work resolves that tension by using tectonic plates as an intermediate preprocessing domain—large enough to contain a plate’s regional drainage structure, but small enough to generate and cache independently of the rest of the world. Because plate boundaries are defined identically for terrain evaluation and river preprocessing, a plate’s river network can be built directly from its geometric domain, which itself does not require sampling the terrain fields to find its borders. Within this geometric domain, a low-resolution sample of the terrain fields is taken to guide downhill river generation. Because rivers are generated independently for each plate, they do not cross plate boundaries. This limitation is mitigated by constructing the continuous terrain fields so that oceans, plateaus, and mountain ranges naturally discourage cross-boundary drainage.

The key contributions of this work are:

- dual plate representation for terrain sampling and river preprocessing, described in Sections 3.2–3.4;
- lazy local river generation and caching, described

in Section 3.7;

- nearest-river regions that accelerate river-segment lookup during chunk generation, described in Section 3.7.

2 Related Works

Related works for fixed domains generally fall into two broad categories: systems that construct rivers first, then the terrain is constructed around them; and systems that construct the terrain first, then the rivers are constructed along this terrain. Learned terrain generation offers a novel approach to world generation in general, and can be extended to infinite domains [13].

2.1 River-First Generation

River-first approaches generate river network structures before the surrounding terrain [10–12]. By treating rivers as the primary organizing structure, these methods guarantee coherent, downhill-flowing rivers while allowing explicit control over river placement, connectivity, and slope. However, because terrain is constructed around a predefined river network, large-scale geological processes such as tectonic uplift and mountain building are secondary to river constraints. Furthermore, these approaches require preprocessing over a fixed domain and are not readily extendable to infinite procedural worlds.

2.2 Terrain-First Generation

Terrain-first approaches first generate terrain before deriving river networks from the resulting topology. Batbold and Ziegelmeier [9] demonstrate that coherent river networks can be derived from procedurally generated terrain using graph-based flow routing. The present work builds on this terrain-first philosophy with river generation derived from a low-resolution sample of terrain fields, which is localized to plate domains. This enables river generation without preprocessing the entire domain.

2.3 Learned Generation

InfiniteDiffusion [13] resolves the infinite-domain limitation of both river-first and terrain-first methods. Current limitations of diffusion-based terrain generators include limited explicit control over large-scale

terrain features, such as continent placement, mountain range orientation, or coastline shape and no guarantee of coherent, downhill-flowing river networks. These limitations are not necessarily fundamental to diffusion models, but do remain unaddressed.

3 Plate-Local River Generation

The following subsections define deterministic plate seeds and their dual pointwise and geometric representations, then describe plate-local terrain sampling, river tracing, caching, and on-demand composition. Together, these stages preserve terrain-aware downhill drainage without a global preprocessing pass. For applications to finite, closed-shaped worlds such as planet domains, we recommend slight modifications to this system, which we provide in this section alongside the infinite application.

Figure 2 summarizes this pipeline, separating the per-plate cache build from on-demand terrain evaluation. Algorithms 1 and 2 detail the complete procedures for building plate-local river caches and evaluating terrain on demand.

Algorithm 1. Plate-Local River Cache Build

Require: Plate i , seed positions $\{\mathbf{c}_j\}$, surface $\mathcal{S}$
Ensure: River network $\mathcal{R}_i$ and nearest-river regions
Geometric plate domain (Section 3.4)
1: Initialize bounding volume B_i around $\mathbf{c}_i$
2: **for** each neighboring plate $j \in \mathcal{N}_i$ **do**
3: $\mathbf{m}_{ij} \leftarrow \frac{1}{2}(\mathbf{c}_i + \mathbf{c}_j)$; $\mathbf{n}_{ij} \leftarrow \mathbf{c}_j - \mathbf{c}_i$
4: Clip B_i by $(\mathbf{p} - \mathbf{m}_{ij}) \cdot \mathbf{n}_{ij} \leq 0$
5: **end for**
6: $\Omega_i^{\mathcal{S}} \leftarrow B_i \cap \mathcal{S}$
Low-resolution plate grid (Section 3.5)
7: **for** each (u, v) with $\mathbf{x}_{uv} \in \Omega_i^{\mathcal{S}}$ **do**
8: $h_{uv} \leftarrow A_{\text{pre}} H_{\text{pre}}(\mathbf{x}_{uv})$
9: **end for**
River node and segment generation (Section 3.6)
10: $\mathcal{N} \leftarrow \emptyset$; $\mathcal{R}_i \leftarrow \emptyset$
11: Select eligible highland sources outside the border mask
12: Add the selected source nodes to $\mathcal{N}$
13: **for** each source node $n \in \mathcal{N}$ **do**
14: Trace downhill from n along grid gradients
15: Stop at sea level, a local minimum, or an existing segment
16: Add the resulting segments to $\mathcal{R}_i$
17: **end for**
18: Assign Strahler orders after establishing all confluences

Nearest-river partitioning (Section 3.7)

- 19: Partition $\Omega_i^{\mathcal{S}}$ into $\{V_s\}_{s \in \mathcal{R}_i}$
20: Cache the network, regions, elevations, and stream orders
-

Algorithm 2. On-Demand Terrain Sample

Require: Position $\mathbf{x}$, seed positions $\{\mathbf{c}_j\}$, river cache
Ensure: Final height $H_{\text{final}}(\mathbf{x})$
Plate assignment (Section 3.3)
1: $\tilde{\mathbf{x}} \leftarrow s_p \mathbf{x} + \mathbf{w}(\mathbf{x})$
2: $i \leftarrow \arg \min_{j \in \mathcal{A}} \|\tilde{\mathbf{x}} - \mathbf{c}_j\|^2$ $\triangleright$ Owing plate; local 3×3 for 2D or $3 \times 3 \times 3$ for 3D lattice search if $\mathcal{A} = \mathbb{Z}^d$
Cache lookup and lazy build (Section 3.7)
3: **if** the cache for plate i is absent **then**
4: Execute Algorithm 1 for plate i
5: **end if**
Pointwise terrain fields (Section 3.3)
6: Evaluate W_{land} , W_{mountain} , W_{plateau} , and W_{island} at $\mathbf{x}$
7: Compute $H_{\text{pre}}(\mathbf{x})$ from the weighted fields
River query (Section 3.7)
8: **if** $\mathcal{R}_i = \emptyset$ **then**
9: **return** $H_{\text{pre}}(\mathbf{x})$
10: **end if**
11: Use the region partition to identify $s^*(\mathbf{x})$
12: $\delta_{\text{river}}(\mathbf{x}) \leftarrow \text{dist}(\mathbf{x}, s^*(\mathbf{x}))$
13: Interpolate $H_{\text{river}}(\mathbf{x})$ along $s^*(\mathbf{x})$
Terrain composition (Section 3.8)
14: $\lambda_{\text{river}}(\mathbf{x}) \leftarrow \left[\min \left(1, \frac{\delta_{\text{river}}(\mathbf{x})}{\omega_{\text{river}}} \right) \right]^{\alpha_{\text{river}}}$
15: $H_{\text{final}}(\mathbf{x}) \leftarrow \lambda_{\text{river}}(\mathbf{x}) H_{\text{pre}}(\mathbf{x}) + (1 - \lambda_{\text{river}}(\mathbf{x})) H_{\text{river}}(\mathbf{x})$
16: **return** $H_{\text{final}}(\mathbf{x})$

3.1 Deterministic Tectonic Plate Positions

Each tectonic plate is defined by a deterministic seed position. The same construction can be used for finite and infinite worlds: finite worlds use a bounded set of plate identifiers, while infinite worlds generate plate identifiers from lattice cells on demand. In both cases, terrain field samples are assigned to the nearest deterministic seed position, producing Voronoi-like plate regions [1, 2].

For an infinite world, plate seeds may be generated as one deterministic point per integer lattice cell, following the same nearest-feature-point structure used by

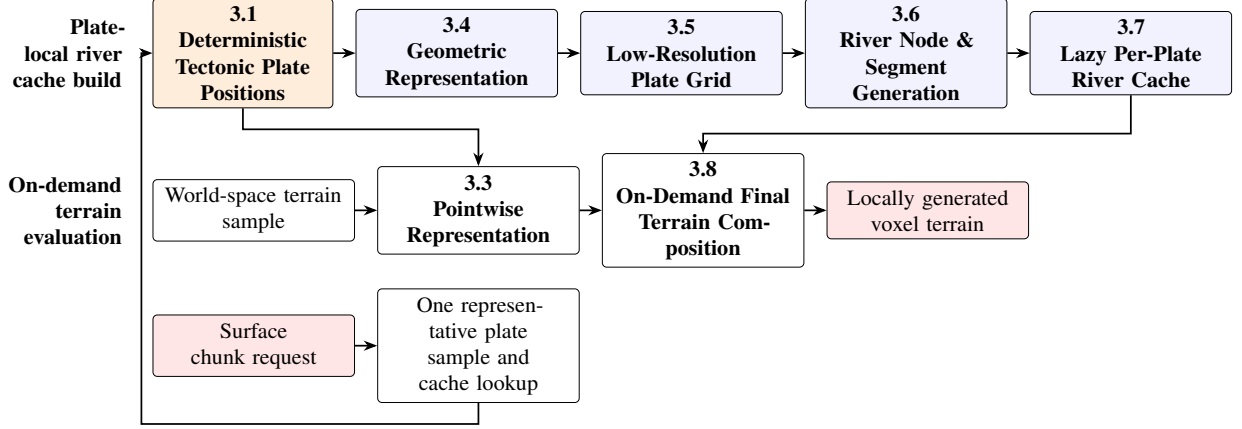


Figure 2: Pipeline overview, generalized for various world types—infinite or finite, and landscape structures—voxels or mesh.

cellular texture bases [6],

$$\mathbf{c}_i = s_p(\mathbf{i} + \boldsymbol{\eta}(\mathbf{i})), \quad \mathbf{i} \in \mathbb{Z}^d, \quad (1)$$

where s_p is the plate scale, i is the plate identifier associated with integer lattice-cell coordinate $\mathbf{i}$, and $\boldsymbol{\eta}(\mathbf{i})$ is a deterministic hash offset. For this present implementation, $\boldsymbol{\eta}(\mathbf{i})$ remains an offset on the 2D plane. Considering an additional offset in 3D space will result in continental landmasses that are less uniform in size, which may be preferred.

For a finite world, with a closed geometry and N plates, a golden-angle sequence gives approximately even seed directions [3, 4]:

$$\theta_i = i\pi(3 - \sqrt{5}), \quad y_i = 1 - \frac{2i+1}{N}, \quad r_i = \sqrt{1 - y_i^2}, \quad (2)$$

$$\mathbf{q}_i = (r_i \cos \theta_i, y_i, r_i \sin \theta_i), \quad (3)$$

$$\mathbf{c}_i = s_p \Pi_{\mathcal{S}}(\mathbf{q}_i) + \boldsymbol{\eta}_i, \quad 0 \leq i < N, \quad (4)$$

Here $\Pi_{\mathcal{S}}$ projects the unit direction onto the chosen finite world surface, and $\boldsymbol{\eta}_i$ is a deterministic jitter applied to the finite plate center.

3.2 Dual Tectonic Plate Representation

The tectonic seed positions are used in two forms: a *pointwise representation*, defined by continuous coordinate-based fields for terrain evaluation, and a *geometric representation*, defined by a bounded non-rasterized domain for regional river preprocessing. The same seed position is used to determine both representations for that given plate.

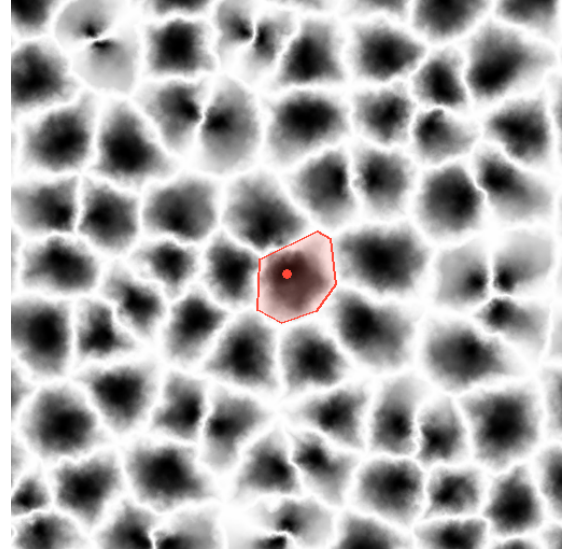


Figure 3: Dual plate representations. A continuous terrain field (grayscale) and a corresponding geometric plate domain (red) are independently derived from the same tectonic plate seed.

Figure 3 shows both representations derived from the same plate seed.

3.3 Pointwise Representation

During chunk generation, terrain is evaluated directly from world-space coordinates by comparing plate seed positions and deriving continuous scalar and vector weight fields from plate ownership, plate type, border distance, and drift direction. These weight fields control land, mountain, plateau, and island chain placement. Noise warping and surface rough-

ness are applied to produce more organic continent shapes [5].

The primary purposes of the pointwise representation are to generate terrain features that appear natural, and to generate terrain features with natural barriers, such as mountains, plateaus, and oceans, at plate borders, providing a natural motive for localizing rivers to each plate and prohibiting plate boundary crossings. Otherwise, the pointwise representation is not essential to this system, and the continuous terrain fields can be constructed to disregard the plate boundaries entirely.

Figure 4 shows various configurations to parameters in the equations defined later in this subsection.

Pointwise Plate. For a world-space point $\mathbf{x}$, the pointwise plate fields are evaluated in warped plate space,

$$\tilde{\mathbf{x}} = s_p \mathbf{x} + \mathbf{w}(\mathbf{x}), \quad (5)$$

where $\mathbf{w}$ is a deterministic noise warp. Note that $\mathbf{w}$ cannot be represented in the geometric representation of the same plate. Given deterministic plate centers $\mathbf{c}_i$, the owning plate is the nearest center in this space:

$$i = \arg \min_{j \in \mathcal{A}} \|\tilde{\mathbf{x}} - \mathbf{c}_j\|^2. \quad (6)$$

Here $\mathcal{A}$ is the set of possible plate identifiers. For a finite world, $\mathcal{A} = \{1, \dots, N\}$ and each identifier names one generated plate seed. For an infinite world, $\mathcal{A} = \mathbb{Z}^d$ and each identifier is an integer lattice-cell coordinate that generates one seed on demand. Rather than searching all of $\mathbb{Z}^d$, the implementation restricts this $\arg \min$ to the 3×3 for 2D or $3 \times 3 \times 3$ for 3D neighborhood of lattice cells surrounding $\tilde{\mathbf{x}}$'s own cell, following the same bounded-jitter neighborhood search used for cellular texture bases [6]. This restriction is exact, independent of the warp $\mathbf{w}$, provided the per-axis hash offset is bounded to at most r cell-widths from the cell center: for a query point at a corner shared by four cells, each of those four seeds can be pushed no farther than $(0.5 + r)\sqrt{2}$ from the corner, while a seed two cells away along either axis can be pulled as close as $\sqrt{(1.5 - r)^2 + (r - 0.5)^2}$. These two distances are equal at $r = 1/3$, so bounding the hash offset to $r \leq 1/3$ guarantees the neighborhood

always contains the true nearest seed; the implementation enforces $r = 1/3$. Plate fields are controlled by the distance margin between the owning plate and a neighboring plate:

$$d_j(\mathbf{x}) = \|\tilde{\mathbf{x}} - \mathbf{c}_j\| - \|\tilde{\mathbf{x}} - \mathbf{c}_i\|, \quad j \neq i. \quad (7)$$

Small values of d_j indicate proximity to the boundary between the owning plate i and neighbor plate j . When a terrain feature depends on the nearest eligible neighboring plate, the margin is minimized over a selected neighbor set $\mathcal{N}$:

$$d_{\mathcal{N}}(\mathbf{x}) = \min_{j \in \mathcal{N}} d_j(\mathbf{x}). \quad (8)$$

Plate Drift. To mimic the effects of tectonic plate drift, where land, mountains, and plateaus are largest near colliding plate borders, we define plate drift vector and scalar fields to guide the magnitudes of subsequent weight fields.

Each plate identifier is assigned a deterministic drift axis $\mathbf{a}_i$ by projecting a random vector $\mathbf{r}_i$ into the tangent plane of the terrain domain. For a non-flat world, such as a finite planet with a closed geometry, the plate center direction vector $\mathbf{c}_i$ from the world origin provides a stable reference normal:

$$\mathbf{a}_i = \frac{\mathbf{c}_i \times \mathbf{r}_i}{\|\mathbf{c}_i \times \mathbf{r}_i\|}. \quad (9)$$

For a flat world, such as an infinite one, the constant reference normal $\mathbf{n}_0$ can be used instead:

$$\mathbf{a}_i = \frac{\mathbf{n}_0 \times \mathbf{r}_i}{\|\mathbf{n}_0 \times \mathbf{r}_i\|}. \quad (10)$$

The drift direction at the terrain sample is then

$$\ell(\mathbf{x}) = \frac{\mathbf{a}_i \times \mathbf{n}(\mathbf{x})}{\|\mathbf{a}_i \times \mathbf{n}(\mathbf{x})\|}, \quad (11)$$

where $\mathbf{n}(\mathbf{x})$ is the local surface normal, and $\ell(\mathbf{x})$ is the local tangent drift direction used at the terrain sample. Note that for a flat world, this greatly simplifies to $\ell(\mathbf{x}) = \mathbf{r}_i$ if $\mathbf{r}_i \in \mathbb{R}^2$.

The scalar drift field, $\rho(\mathbf{x})$, measures signed progress along the local drift direction:

$$\rho(\mathbf{x}) = (\tilde{\mathbf{x}} - \mathbf{c}_i) \cdot \ell(\mathbf{x}). \quad (12)$$

Weights. Land, island, and mountain feature weights use the same normalized beta-shaped border profile. For feature k with amplitude A_k , width ω_k , interior exponent p_k , border exponent q_k , and plate-coverage parameter γ_k ,

$$C_k = \left[\left(\frac{p_k}{p_k + q_k} \right)^{p_k} \left(\frac{q_k}{p_k + q_k} \right)^{q_k} \right]^{-1}, \quad (13)$$

$$D_k(\mathbf{x}) = \max \left(0, \frac{\rho(\mathbf{x}) + 2\gamma_k - 1}{2\gamma_k} \right), \quad (14)$$

$$z_k(\mathbf{x}) = \frac{d_{\mathcal{N}_k}(\mathbf{x})}{\omega_k}, \quad (15)$$

$$W_k(\mathbf{x}) = \begin{cases} A_k C_k D_k(\mathbf{x}) z_k(\mathbf{x})^{p_k} \cdot (1 - z_k(\mathbf{x}))^{q_k}, & 0 \leq z_k(\mathbf{x}) \leq 1, \\ 0, & \text{otherwise.} \end{cases} \quad (16)$$

The coefficient C_k normalizes the profile before amplitude scaling, and D_k gates the feature by the drift field.

Plateaus are treated separately because, in this work, we define them as highlands arising from converging continental plates rather than a single boundary-distance profile. For a nearby continental pair (i, j) , let $\hat{\mathbf{u}}_{ij}(\mathbf{x})$ denote the unit separation direction from plate i to plate j after projection into the local tangent plane. The closing response is measured by projecting each plate's tangent drift onto this direction:

$$\kappa_{i \rightarrow j}(\mathbf{x}) = \begin{cases} (\mathbf{a}_i \times \mathbf{n}(\mathbf{x})) \cdot \hat{\mathbf{u}}_{ij}(\mathbf{x}), & \text{if positive,} \\ 0, & \text{if zero or negative,} \end{cases} \quad (17)$$

$$\kappa_{j \rightarrow i}(\mathbf{x}) = \begin{cases} -(\mathbf{a}_j \times \mathbf{n}(\mathbf{x})) \cdot \hat{\mathbf{u}}_{ij}(\mathbf{x}), & \text{if positive,} \\ 0, & \text{if zero or negative.} \end{cases} \quad (18)$$

These detect colliding plates. The weight itself is constructed as,

$$R_{\text{plateau}}(\mathbf{x}) = O(\mathbf{x}) \max_{(i,j) \in \mathcal{C}} B_{ij}(\mathbf{x}) N_i(\mathbf{x}) \cdot \kappa_{i \rightarrow j}(\mathbf{x}) \kappa_{j \rightarrow i}(\mathbf{x}) \cdot S\left(\frac{\kappa_{i \rightarrow j}(\mathbf{x})}{\eta}\right) S\left(\frac{\kappa_{j \rightarrow i}(\mathbf{x})}{\eta}\right), \quad (19)$$

$$W_{\text{plateau}}(\mathbf{x}) = A_{\text{plateau}} S(g_{\text{plateau}} R_{\text{plateau}}(\mathbf{x})) \cdot (A_{\text{land}} - W_{\text{land}}(\mathbf{x})) \cdot (A_{\text{mountain}} - W_{\text{mountain}}(\mathbf{x})). \quad (20)$$

Here $\mathcal{C}$ is the set of nearby continental plate pairs, B_{ij} fades with the distance between the plate centers, N_i fades with the distance from the owning plate, O suppresses plateau formation near oceanic boundaries, η is the closing fade width, g_{plateau} controls the plateau response gain, and S is a smoothstep function. The final plateau weight W_{plateau} is an amplitude-scaled smoothstep response to the closing response R_{plateau} , suppressed where the land and mountain fields already dominate.

Pre-River Terrain Height Composition. The pre-river terrain height composition, $H_{\text{pre}}(\mathbf{x})$ is the simple sum of the land, island, mountain, and plateau weight fields. An addition constant weight field may be used for augmenting the height floor of continental plates versus oceanic plates, which are defined by the presence or absence of this floor.

3.4 Geometric Representation

The entire plate domain can be constructed geometrically from nearby seed positions without terrain field sampling. Because the noise warping applied to the weight fields cannot be reproduced geometrically, this domain is a straight-bordered approximation of the true plate footprint—close enough for river generation purposes.

The geometric plate domain is initialized as a bounded volume B_i around the plate center and then clipped by the bisector planes between $\mathbf{c}_i$ and $\mathbf{c}_j$:

$$\Omega_i = B_i \cap \bigcap_{j \neq i} \{ \mathbf{p} : (\mathbf{p} - \mathbf{m}_{ij}) \cdot \mathbf{n}_{ij} \leq 0 \}, \quad (21)$$

where

$$\mathbf{m}_{ij} = \frac{1}{2}(\mathbf{c}_i + \mathbf{c}_j), \quad (22)$$

$$\mathbf{n}_{ij} = \mathbf{c}_j - \mathbf{c}_i. \quad (23)$$

Note that $j \in \mathcal{N}_i$, where $\mathcal{N}_i$ is finite. For a world surface $\mathcal{S}$, the plate-local preprocessing domain is the surface cross section of this clipped volume,

$$\Omega_i^{\mathcal{S}} = \Omega_i \cap \mathcal{S}. \quad (24)$$

The Sutherland-Hodgman polygon clipping algorithm is used for this step in the implementation [7]. Recall

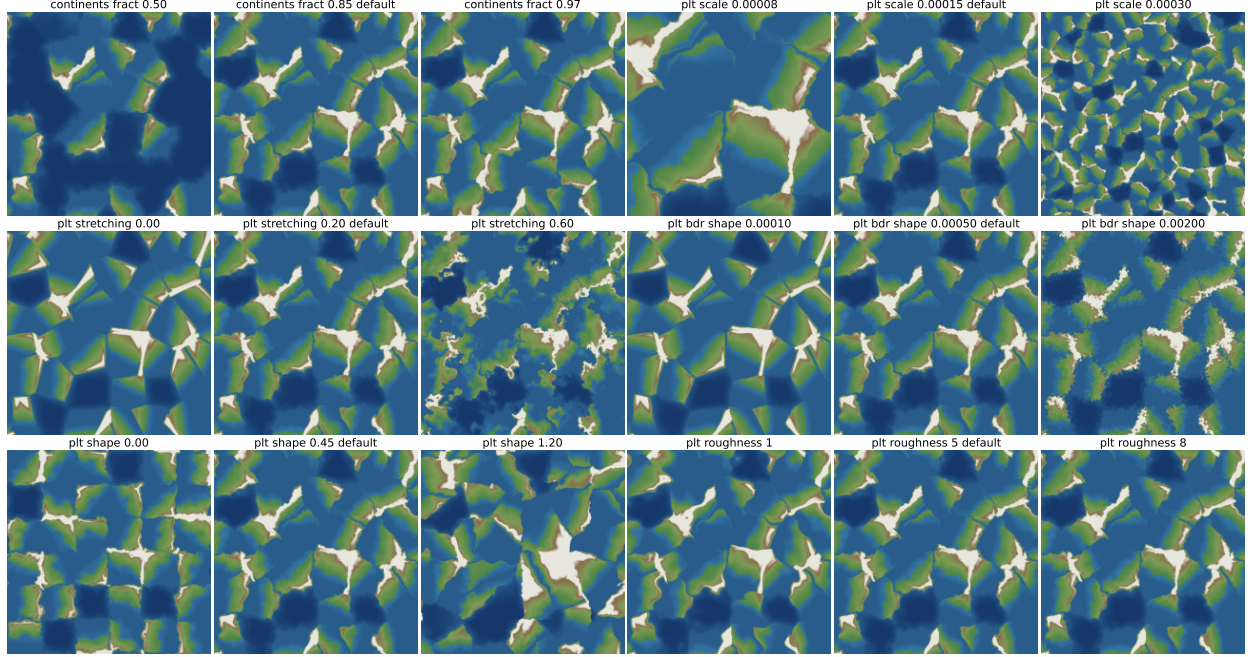


Figure 4: Top-down height-field renders for one fixed seed, sweeping one plate-tessellation parameter per row (low, default, high) with all other parameters held at their Table-listed defaults. Row 1: oceanic/continental threshold, then plate scale s_p . Row 2: warp amplitude, then warp spatial frequency. Row 3: plate-seed jitter (zero jitter collapses to a regular lattice), then warp octave count.

that this geometric representation is not identical to the pointwise representation due to the warp term $\mathbf{w}$ in Equation (5).

3.5 Low-Resolution Plate Grid

The river generator samples pre-river terrain using the weight fields on a low-resolution grid restricted to the geometric representation domain, providing approximate terrain elevation and slope for river path planning. Using a low-resolution terrain sample makes plate-local river preprocessing computationally practical by significantly reducing computation time and memory requirements, while still retaining a terrain-first generation philosophy.

For each accepted grid sample (u, v) on plate i , the sample position lies in the geometric surface domain and stores the pre-river terrain height:

$$\begin{aligned} \mathbf{x}_{uv} &\in \Omega_i^{\mathcal{S}}, \\ h_{uv} &= A_{\text{pre}} H_{\text{pre}}(\mathbf{x}_{uv}), \end{aligned} \quad (25)$$

where $A_{\text{pre}} < 1$ is a pre-river height attenuation factor. By reducing the terrain height used for river planning, the subsequently enforced river elevations are

biased below the surrounding terrain surface, producing valley-like depressions after composition. A_{pre} may be defined to attenuate land, mountain, plateau, and island features differently.

3.6 River Node & Segment Generation

River source nodes are selected from grid points of sufficient elevation. Segments are traced downhill along the grid until reaching sea level, a local minimum, or an existing river segment, producing outlets, lakes, or confluences respectively.

Note that the algorithm for placing nodes and segments is arbitrary and can be modified or replaced without affecting the rest of the pipeline. However, to avoid blending discontinuities during final terrain composition, nodes and segments should not be placed near plate borders. The river blending width, ω_{river} (see Section 3.8), defines a proximity mask for removing or preventing node placement near these borders. Because river-terrain blending does not extend beyond ω_{river} from a given river, this constraint prevents river blending from crossing plate borders and introducing discontinuities.

3.7 Lazy Per-Plate River Cache

A plate’s river network is generated the first time a chunk samples terrain from that plate and finds the cache absent. The cache stores river nodes, segments, and elevations, along with a non-rasterized geometric partitioning of the plate domain into nearest-river regions. These nearest-river regions allow chunk generation to locate the nearest river segment faster. Measured directly against a brute-force per-segment distance scan over the same cached networks, this partition-based lookup is $2.9\text{--}3.2\times$ faster across plates containing 60–234 river segments, with the advantage widening as segment count grows (Section 4.1). Because rivers naturally should not cross tectonic boundaries—where plateaus and oceans generate within this system—each plate’s network can be generated and cached in isolation without triggering generation of neighboring plates.

For a terrain sample on plate i , let $\mathcal{R}_i$ be the cached set of river segments for that plate. The nearest segment and its distance are

$$\begin{aligned} s^*(\mathbf{x}) &= \arg \min_{s \in \mathcal{R}_i} \text{dist}(\mathbf{x}, s), \\ \delta_{\text{river}}(\mathbf{x}) &= \text{dist}(\mathbf{x}, s^*(\mathbf{x})). \end{aligned} \quad (26)$$

Additional river fields can be evaluated from the same nearest segment. In particular, the Strahler stream order [8] provides a deterministic river-size field by assigning source tributaries order one and increasing the order at confluences where equal-order tributaries meet. For a sample $\mathbf{x}$, the size field may therefore be taken from the nearest segment,

$$\sigma_{\text{river}}(\mathbf{x}) = \text{strahler}(s^*(\mathbf{x})). \quad (27)$$

Small deterministic noise offsets may also be applied to the river distance, height, or size fields before final composition. When the same seed and coordinate basis are used for all chunks, these perturbations make river channels appear more curved while preserving deterministic local generation.

3.8 On-Demand Final Terrain Composition

During chunk generation, plate weight fields are sampled, distance to the nearest river segment is computed, and river elevation $H_{\text{river}}(\mathbf{x})$ is interpolated between nodes. The river distance is converted into a

normalized terrain blend weight before mixing the local terrain height with the river height:

$$\begin{aligned} \lambda_{\text{river}}(\mathbf{x}) &= \left[\min \left(1, \frac{\delta_{\text{river}}(\mathbf{x})}{\omega_{\text{river}}} \right) \right]^{\alpha_{\text{river}}}, \\ H_{\text{final}}(\mathbf{x}) &= \lambda_{\text{river}}(\mathbf{x}) H_{\text{pre}}(\mathbf{x}) + (1 - \lambda_{\text{river}}(\mathbf{x})) H_{\text{river}}(\mathbf{x}). \end{aligned} \quad (28)$$

Here ω_{river} is the river blending width and α_{river} controls the blend falloff. Near the river, λ_{river} approaches zero and the interpolated river height dominates; away from the river, λ_{river} approaches one and the locally generated terrain is preserved.

4 Experimental Results and Evaluation

A video demonstration for a finite, cube-shaped planet implementation of this system is available on YouTube at youtube.com/watch?v=4Ai_13znvgg.

Code for a toy example similar to Figure 1 is available at github.com/Davismk3/plate-local-river-generation.

Figure 1 and Table 2 show output and measured performance for a single deterministic run, detailed below. All benchmark code and raw results accompany this submission; every statistic is computed directly from the released flat-infinite implementation.

The implementation is written in Python with OpenGL and PyGame rendering. No game engine is used. Hot functions use Numba @njit [14]; additional optimizations are used elsewhere.

4.1 Nearest-Segment Lookup Ablation

Section 3.7 argues that the nearest-river region partition lets terrain sampling locate a query point’s nearest river segment faster than without it. To quantify that claim, we compare the region-based lookup against a brute-force baseline that instead scans every cached segment in the plate directly; both feed the same underlying distance and height interpolation, so only the lookup strategy differs.

Both lookups were measured as compiled (@njit) loops over the same 320×320 (102,400-sample) terrain grid used in the reference configuration (Sec-

tion 4.2), against the same plate and terrain, sweeping the requested river-path count to vary the accepted segment count from 60 to 234 (the plate saturates at 234 segments under the reference configuration’s other generation parameters, regardless of how many additional paths are requested). Table 1 reports the fastest of five runs per configuration.

The region-based lookup is consistently $2.9\text{--}3.2\times$ faster than the brute-force scan, and the margin widens as segment count grows. Both lookups are linear scans over the plate’s segments or regions, but the region lookup typically terminates after testing only the region containing the query point, while brute force always evaluates every segment’s distance. Ablation code and full results accompany this submission alongside the reference-configuration benchmarks.

Segments	Region (ms)	Brute force (ms)	Speedup
60	92.4	270.9	$2.93\times$
120	168.7	530.8	$3.15\times$
177	243.6	787.9	$3.23\times$
234	323.3	1042.2	$3.22\times$

Table 1: Nearest-segment lookup time over a 102,400-point terrain grid, region-based vs. brute-force, at increasing plate segment counts.

4.2 Reference Configuration

The figure exporter also returns a JSON summary and a CSV file from the same run. All reported measurements correspond to Figure 1. The figure is not generated separately. The run uses seed 1 and a square view two plate-cell widths across. The terrain is sampled at a resolution of 320×320 (102,400 samples). Ten plate caches intersect the view. River networks are generated on a 100×100 grid within each plate. At most 50 river paths are requested per plate. Channel lengths are normalized to plate-cell widths.

The sampled terrain contains 60.93% land and 39.07% ocean at a sea level of 0.35. Terrain heights range from 0.200 to 1.449. The median height is 0.439. The 95th percentile height is 1.200.

The 10 plate caches contain 1,992 nodes and 1,807 segments. Of the 500 requested river paths, 408 are accepted (81.6%). The networks contain 397 source

nodes, 170 sea outlets, 16 local-minimum outlets, 185 confluences, and 186 connected components.

The total channel length across all plate caches is 58.084 plate-cell widths. The average channel length per plate is 5.808 plate-cell widths. Individual plates contain 141–230 segments and 28–49 accepted river paths. Their total channel lengths range from 4.647 to 7.521 plate-cell widths.

Every stored river segment was tested using its adjusted endpoint river heights. No uphill segments were found. Downhill compliance is 100%.

The nearest-segment partition contains 1,807 regions, one for each river segment. These regions are represented by 10,453 polygon vertices in total.

After clipping the river network to the meshed domain shown in Figure 1, 1,055 nodes and 925 segments remain. The visible network contains 239 source nodes, 80 sea outlets, 14 local-minimum outlets, and 722 channel nodes. The visible channel length is 30.089 plate-cell widths.

The packed numeric cache for the 10 plate caches occupies 0.488 MiB. This corresponds to approximately 283 bytes per river segment. Python object overhead used during preprocessing is excluded.

The measurements were collected on an Apple M4 Max system. Construction of the 10 plate caches requires 4.628s in total. The average construction time is 0.463s per cache. The median is 0.466s. Individual cache construction times range from 0.251 to 0.674s. Cache packing requires 0.008s. Terrain sampling and visible-geometry generation require 0.404s. Rendering and PNG export require 1.764s. Rendering includes $3\times$ supersampling for anti-aliasing, down-sampling to the final image resolution, and rasterization of an auxiliary depth buffer for terrain occlusion. These timings are observational rather than controlled benchmarks. They may vary with compilation state and system load. The terrain and river network statistics are deterministic.

Table 2. Metrics emitted by the same deterministic exporter invocation that produced Figure 1. Lengths are measured in plate-cell widths. Complete-network counts include all 10 plate caches intersecting the view; visible counts are clipped to the displayed mesh.

Metric	Value
Terrain samples	102,400
Visible plate caches	10
Land / ocean fraction	60.93 / 39.07%
Median / 95th-percentile height	0.439 / 1.200
Nodes / segments	1,992 / 1,807
Accepted / requested paths	408 / 500
Sources / confluences	397 / 185
Sea / local-minimum outlets	170 / 16
Connected components	186
Downhill violations	0
Total / visible channel length	58.084 / 30.089
Visible nodes / segments	1,055 / 925
Regions / polygon vertices	1,807 / 10,453
Packed-cache size	0.488 MiB
Cache build total / mean	4.628 / 0.463 s
Pack / sample / render	0.008 / 0.404 / 1.764 s

5 Future Work

We recommend that future implementations of this system be written in a compiled language like C++, not Python. Python was selected for its familiarity at the time.

The plate-local framework is not limited to rivers. The same bounded-domain pattern could generate roads, towns, or other features that guarantee a certain topology. Each application would replace downhill flow with its own constraints—for example slope and travel cost for roads, or water access and terrain suitability for towns—while retaining deterministic identifiers, lazy regional preprocessing, packed caches, and point-wise evaluation.

Most notably, integration with learned generators such as InfiniteDiffusion [13] offers a longer-term hybrid direction. Plate-local fields and river graphs could supply explicit structural constraints, while a learned model adds fine surface, coastline, and mountain detail.

6 Conclusion

We have presented an algorithm for lazily generating river networks that are terrain-aware, deterministic, and guarantee coherence and downhill flow, which can be tailored to flat infinite world-types or closed-shaped finite world-types such as planets.

References

- [1] F. Aurenhammer, “Voronoi diagrams—a survey of a fundamental geometric data structure,” *ACM Computing Surveys*, vol. 23, no. 3, pp. 345–405, 1991.
- [2] A. Okabe, B. Boots, K. Sugihara, and S. N. Chiu, *Spatial Tessellations: Concepts and Applications of Voronoi Diagrams*, 2nd ed. Wiley, 2000.
- [3] E. B. Saff and A. B. J. Kuijlaars, “Distributing many points on a sphere,” *The Mathematical Intelligencer*, vol. 19, pp. 5–11, 1997. doi: [10.1007/BF03024331](https://doi.org/10.1007/BF03024331).
- [4] R. Swinbank and R. James Purser, “Fibonacci grids: A novel approach to global modelling,” *Quarterly Journal of the Royal Meteorological Society*, vol. 132, pp. 1769–1793, 2006. doi: [10.1256/qj.05.227](https://doi.org/10.1256/qj.05.227).
- [5] K. Perlin, “An image synthesizer,” *ACM SIGGRAPH Computer Graphics*, vol. 19, no. 3, pp. 287–296, 1985.
- [6] S. Worley, “A cellular texture basis function,” in *Proceedings of SIGGRAPH 1996*, pp. 291–294, 1996.
- [7] I. E. Sutherland and G. W. Hodgman, “Reentrant polygon clipping,” *Communications of the ACM*, vol. 17, no. 1, pp. 32–42, 1974.
- [8] A. N. Strahler, “Quantitative analysis of watershed geomorphology,” *Eos, Transactions American Geophysical Union*, vol. 38, no. 6, pp. 913–920, 1957. doi: [10.1029/TR038i006p00913](https://doi.org/10.1029/TR038i006p00913).
- [9] B. Batbold and L. Ziegelmeier, “From chaos to continents: Voronoi-based procedural terrain generation with hydrology and 3D visualization,” in *42nd International Symposium on Computational Geometry (SoCG 2026)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 367. Schloss Dagstuhl—Leibniz-Zentrum für Informatik, 2026, pp. 101:1–101:7. doi: [10.4230/LIPIcs.SocG.2026.101](https://doi.org/10.4230/LIPIcs.SocG.2026.101).
- [10] J.-D. Génevaux, É. Galin, E. Guérin, A. Peytavie, and B. Benes, “Terrain generation using procedural models based on hydrology,” *ACM Transactions on Graphics*, vol. 32, no. 4, pp. 143:1–143:13, 2013. doi: [10.1145/2461912.2461996](https://doi.org/10.1145/2461912.2461996).
- [11] E. Derzapf, B. Ganster, M. Guthe, and R. Klein, “River networks for instant procedural planets,” *Computer Graphics Forum*, vol. 30, no. 7, pp. 2031–2040, 2011.
- [12] R. Fischer, J. Boeckers, and G. Zachmann, “Procedural generation of landscapes with water bodies using artificial drainage basins,” in *Advances in Computer Graphics (CGI 2022)*, Lecture Notes in Computer Science, vol. 13443. Springer, 2022, pp. 345–356. doi: [10.1007/978-3-031-23473-6_27](https://doi.org/10.1007/978-3-031-23473-6_27).
- [13] A. Goslin, “InfiniteDiffusion: Bridging learned fidelity and procedural utility for open-world terrain generation,” in *SIGGRAPH Conference Papers ’26*. New York, NY, USA: ACM, 2026, 10 pages. doi: [10.1145/3799902.3811080](https://doi.org/10.1145/3799902.3811080).
- [14] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A LLVM-based Python JIT compiler,” in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 2015. doi: [10.1145/2833157.2833162](https://doi.org/10.1145/2833157.2833162).